

✓ Part 1: The Code Blocks for Your Smallest First Version

This implementation runs your Non-AI Baseline and your Gated LLM System side-by-side. It implements the exact Python Span-Containment Guardrail and Abstention Logic required by your professor to hit the 35% implementation block.

```
# Install the official Google GenAI SDK and Pydantic for rigid schema control
!pip install google-genai pydantic

import re
import json
from google import genai
from google.genai import types
from pydantic import BaseModel, Field
from google.colab import userdata

# Initialize the Gemini Client using your Colab Secrets Key
# (Make sure to add 'GEMINI_API_KEY' to your Colab Secrets tab on the left)
try:
    api_key = userdata.get('GEMINI_API_KEY')
    client = genai.Client(api_key=api_key)
except Exception as e:
    print("API Key missing! Please add GEMINI_API_KEY to your Colab Secrets.")
```

```
Requirement already satisfied: google-genai in /usr/local/lib/python3.13/dist-packages (2.12.1)
Requirement already satisfied: pydantic in /usr/local/lib/python3.13/dist-packages (2.13.5)
Requirement already satisfied: anyio<5.0.0,>=4.8.0 in /usr/local/lib/python3.13/dist-packages (from google-genai) (4.8.0)
Requirement already satisfied: google-auth<2.56.0,>=2.48.1 in /usr/local/lib/python3.13/dist-packages (from google-genai) (2.48.1)
Requirement already satisfied: httpx<1.0.0,>=0.28.1 in /usr/local/lib/python3.13/dist-packages (from google-genai) (0.28.1)
Requirement already satisfied: requests<3.0.0,>=2.28.1 in /usr/local/lib/python3.13/dist-packages (from google-genai) (2.32.3)
Requirement already satisfied: tenacity<9.2.0,>=8.2.3 in /usr/local/lib/python3.13/dist-packages (from google-genai) (8.5.0)
Requirement already satisfied: websockets<17.0,>=13.0.0 in /usr/local/lib/python3.13/dist-packages (from google-genai) (13.1)
Requirement already satisfied: typing-extensions<5.0.0,>=4.14.0 in /usr/local/lib/python3.13/dist-packages (from google-genai) (4.14.0)
```

```

Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.13/dist-packages (from google-genai)
Requirement already satisfied: sniffio in /usr/local/lib/python3.13/dist-packages (from google-genai) (1.3.1)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.13/dist-packages (from pydantic)
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.13/dist-packages (from pydantic)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.13/dist-packages (from pydantic)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.13/dist-packages (from anyio<5.0.0,>=4.8.0)
Requirement already satisfied: pyasn1-modules>=0.2.1 in /usr/local/lib/python3.13/dist-packages (from google-auth)
Requirement already satisfied: cryptography>=38.0.3 in /usr/local/lib/python3.13/dist-packages (from google-auth)
Requirement already satisfied: rsa<5,>=3.1.4 in /usr/local/lib/python3.13/dist-packages (from google-auth)
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-packages (from httpx<1.0.0,>=0.28.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-packages (from httpx<1.0.0,>=0.28.1)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packages (from httpcore==1.*->httpx)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.13/dist-packages (from requests)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.13/dist-packages (from requests)
Requirement already satisfied: cffi>=2.0.0 in /usr/local/lib/python3.13/dist-packages (from cryptography>=38.0.3)
Requirement already satisfied: pyasn1<0.7.0,>=0.6.1 in /usr/local/lib/python3.13/dist-packages (from pyasn1-modules)
Requirement already satisfied: pycparser in /usr/local/lib/python3.13/dist-packages (from cffi>=2.0.0->cryptography)

```

```

import os, getpass
from google import genai

def _get_gemini_key():
    # Check standard environment variables first
    k = os.environ.get('GEMINI_API_KEY')
    if k: return k

    # Attempt to retrieve from Google Colab Secrets
    try:
        from google.colab import userdata
        k = userdata.get('GEMINI_API_KEY')
        if k: return k
    except Exception:
        pass

    # Fallback to manual entry
    return getpass.getpass('Paste your Gemini API key (hidden): ')

# Securely load the key and initialize the official Google GenAI Client

```

```
API_KEY = _get_gemini_key()
client = genai.Client(api_key=API_KEY)
print('Client initialized successfully!')
```

Client initialized successfully!

```
# A real-world mock text string containing negation and formatting anomalies
mock_consult_note = """
CHIEF COMPLAINT: Patient presents with persistent cough.
PAST MEDICAL HISTORY: Hypertension, managed well. Mother has diabetes.
MEDICATIONS: Patient was on Metformin 500mg until March, but discontinued it.
Today, I am starting the patient on Amoxicillin 250mg TDS for 7 days due to suspected bacterial infection.
"""
print("Mock Consult Note Initialized.")
```

Mock Consult Note Initialized.

```
if 'mock_consult_note' in globals():
    print("✅ Variable 'mock_consult_note' is defined.")
    print("Content length:", len(mock_consult_note))
    print("Preview:", mock_consult_note[:50] + "...")
else:
    print("❌ Variable 'mock_consult_note' is NOT defined. Please run the cell above first.")
```

✅ Variable 'mock_consult_note' is defined.
Content length: 314
Preview:
CHIEF COMPLAINT: Patient presents with persistent...

```
def run_non_ai_baseline(text: str) -> dict:
    """
    Naively extracts common medications using deterministic RegEx.
    Acts as your committed course baseline to measure AI against.
    """
    # Simple list representing an absolute baseline drug dictionary
```

```

gazetteer = ["metformin", "amoxicillin", "lisinopril", "aspirin"]
extracted = {}

# Simple regex pattern to capture standard dosage formatting (digits followed by mg/ml)
dosage_pattern = r'(\d+(?:\s*mg|\s*ml))'

for drug in gazetteer:
    # Case-insensitive search for the drug name
    if re.search(rf"\b{drug}\b", text, re.IGNORECASE):
        # Try to grab the closest dosage window around that word
        match_window = re.search(rf"{drug}\s*\d+\s*mg|(\d+\s*mg)\s*{drug}", text, re.IGNORECASE)
        dosage = re.search(dosage_pattern, match_window.group(0), re.IGNORECASE).group(0) if match_window
        extracted[drug.capitalize()] = dosage

return extracted

baseline_results = run_non_ai_baseline(mock_consult_note)
print("--- NON-AI BASELINE RESULTS ---")
print(json.dumps(baseline_results, indent=2))
# Note how the baseline incorrectly flags Metformin despite the negation!

```

```

--- NON-AI BASELINE RESULTS ---
{
  "Metformin": "500mg",
  "Amoxicillin": "250mg"
}

```

```

class MedicationExtraction(BaseModel):
    # We strictly mandate that the model provides the exact source phrase
    drug_name: str = Field(description="The generic or brand name of the active, currently prescribed medication.")
    dosage: str = Field(description="The dosage and frequency of the active medication.")
    verbatim_source_phrase: str = Field(description="The EXACT, literal words from the text that justify this extraction.")

class MedicalSchema(BaseModel):
    # A list wrapper ensuring structured JSON array formatting

```

```
medications: list[MedicationExtraction] = Field(description="List of currently active medications being |
```

```
def run_mediextract_pipeline(text: str) -> dict:
    """
    Executes a single end-to-end path: feeds text to Gemini, enforces
    structured JSON, and applies deterministic Python string containment guardrails.
    """
    prompt = f"""
    You are an expert clinical notation assistant. Analyze the following dictated consult note.
    Extract ONLY the medications that are currently active, being started, or actively continued today.

    CRITICAL SAFETY RULES:
    1. Do NOT extract historical medications that were discontinued or stopped.
    2. Do NOT extract medications belonging to family members (e.g., mother's history).
    3. The 'verbatim_source_phrase' must match characters in the input text EXACTLY, word-for-word.

    Consult Note:
    \"{text}\"
    """

    # 1. Rent the Semantic Reasoning (Gemini 3.5 Flash)
    # Updated model ID to 'gemini-3.5-flash' to resolve the 404 deprecation error
    response = client.models.generate_content(
        model='gemini-3.5-flash',
        contents=prompt,
        config=types.GenerateContentConfig(
            response_mime_type="application/json",
            response_schema=MedicalSchema,
            temperature=0.0 # Force deterministic output processing
        ),
    )

    raw_json = json.loads(response.text)
    guarded_output = []

    # 2. Python Deterministic Guardrail Layer (OWN)
```

```
for entry in raw_json.get("medications", []):
    source_phrase = entry.get("verbatim_source_phrase", "")

    # Check if the text actually contains the exact quote provided by the LLM
    if source_phrase and (source_phrase in text):
        # Verification Pass
        guarded_output.append(entry)
    else:
        # Hallucination Caught / Gated Abstention
        # Force the field to blank/null rather than allowing an ungrounded guess
        print(f"[GUARDRAIL WARNING] Catching Hallucination! Evidence phrase '{source_phrase}' not found :")
        entry["drug_name"] = "BLANK (Abstained: Ungrounded)"
        entry["dosage"] = "BLANK"
        guarded_output.append(entry)

return guarded_output

try:
    ai_results = run_mediextract_pipeline(mock_consult_note)
    print("\n--- MEDIEXTRACT PROTOTYPE RESULTS ---")
    print(json.dumps(ai_results, indent=2))
except Exception as e:
    print(f"An error occurred: {e}")
    print("Please double-check that your GEMINI_API_KEY is correctly set in your Colab Secrets tab.")
```

```
--- MEDIEXTRACT PROTOTYPE RESULTS ---
[
  {
    "drug_name": "Amoxicillin",
    "dosage": "250mg TDS for 7 days",
    "verbatim_source_phrase": "Amoxicillin 250mg TDS for 7 days"
  }
]
```

✓ Part 2: Accuracy Metrics and Performance Comparison

To meet the project requirements, we need to quantitatively compare the two methods. We define a 'Ground Truth' based on clinical validity: only active medications should be extracted.

```
def evaluate_performance(baseline, ai, ground_truth_set):
    """
    Calculates precision, recall, and F1-score based on the active medications list.
    """
    def calculate_stats(results_keys, gt):
        tp = len(results_keys.intersection(gt))
        fp = len(results_keys - gt)
        fn = len(gt - results_keys)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

        return {"Precision": precision, "Recall": recall, "F1-Score": f1}

    # Ground Truth: Only Amoxicillin is active today
    gt = set(ground_truth_set)

    # Normalize keys to title case for comparison
    baseline_keys = set([k.capitalize() for k in baseline.keys()])
    ai_keys = set([entry['drug_name'].capitalize() for entry in ai if "BLANK" not in entry['drug_name']])

    print(f"--- EVALUATION METRICS ---")
    print(f"Baseline Stats: {calculate_stats(baseline_keys, gt)}")
    print(f"AI System Stats: {calculate_stats(ai_keys, gt)}")

    # Define the correct active medications based on clinical review of the note
    ground_truth = ["Amoxicillin"]

    evaluate_performance(baseline_results, ai_results, ground_truth)
```

```
--- EVALUATION METRICS ---
Baseline Stats: {'Precision': 0.5, 'Recall': 1.0, 'F1-Score': 0.6666666666666666}
```

```
AI System Stats: {'Precision': 1.0, 'Recall': 1.0, 'F1-Score': 1.0}
```

```
def evaluate_performance(baseline, ai, ground_truth_set):
    """
    Calculates precision, recall, and F1-score based on the active medications list.
    """
    def calculate_stats(results_keys, gt):
        tp = len(results_keys.intersection(gt))
        fp = len(results_keys - gt)
        fn = len(gt - results_keys)

        precision = tp / (tp + fp) if (tp + fp) > 0 else 0
        recall = tp / (tp + fn) if (tp + fn) > 0 else 0
        f1 = 2 * (precision * recall) / (precision + recall) if (precision + recall) > 0 else 0

        return {"Precision": precision, "Recall": recall, "F1-Score": f1}

    # Ground Truth: Only Amoxicillin is active today
    gt = set(ground_truth_set)

    # Normalize keys to title case for comparison
    baseline_keys = set([k.capitalize() for k in baseline.keys()])
    ai_keys = set([entry['drug_name'].capitalize() for entry in ai if "BLANK" not in entry['drug_name']])

    print("--- EVALUATION METRICS ---")
    print(f"Baseline Stats: {calculate_stats(baseline_keys, gt)}")
    print(f"AI System Stats: {calculate_stats(ai_keys, gt)}")

    try:
        # Define the correct active medications based on clinical review of the note
        ground_truth = ["Amoxicillin"]
        # Run the evaluation with the updated results
        evaluate_performance(baseline_results, ai_results, ground_truth)
    except NameError as e:
        print(f"Could not evaluate metrics yet: {e}")
        print("Please make sure you run the pipeline cell (x3ioB29LSJbG) successfully first to generate the 'ai_")
```

```
--- EVALUATION METRICS ---
```

```
Baseline Stats: {'Precision': 0.5, 'Recall': 1.0, 'F1-Score': 0.6666666666666666}
```

```
AI System Stats: {'Precision': 1.0, 'Recall': 1.0, 'F1-Score': 1.0}
```

```
# 1. Re-initialize the client with the updated Secret key
```

```
try:
```

```
    api_key = userdata.get('GEMINI_API_KEY')
```

```
    client = genai.Client(api_key=api_key)
```

```
    print("Client successfully updated with new API key.")
```

```
except Exception as e:
```

```
    print(f"Could not retrieve API key: {e}")
```

```
# 2. Re-run the extraction pipeline with your working credentials
```

```
try:
```

```
    print("Running extraction on mock consult note...")
```

```
    ai_results = run_mediextract_pipeline(mock_consult_note)
```

```
    print("AI extraction successful!")
```

```
# 3. Automatically run evaluation metrics
```

```
evaluate_performance(baseline_results, ai_results, ground_truth)
```

```
except Exception as e:
```

```
    print(f"Pipeline execution failed: {e}")
```

```
    print("If you are still getting an error, please ensure your model name is 'gemini-3.5-flash'.")
```

```
Client successfully updated with new API key.
```

```
Running extraction on mock consult note...
```

```
AI extraction successful!
```

```
--- EVALUATION METRICS ---
```

```
Baseline Stats: {'Precision': 0.5, 'Recall': 1.0, 'F1-Score': 0.6666666666666666}
```

```
AI System Stats: {'Precision': 1.0, 'Recall': 1.0, 'F1-Score': 1.0}
```

Technical System Summary for LLM Context

```
# SYSTEM SUMMARY & CONTEXT ARCHITECTURE
```

```
## 1. Project Goal & Architecture
- **Objective:** Side-by-side comparison of a deterministic Non-AI Baseline vs. a Gated LLM System for medical e
- **Domain:** Clinical Consult Note parsing (specifically extracting active, currently prescribed medications wh
- **Frameworks:** Python, Google GenAI SDK (`google-genai`), Pydantic (v2) for rigid schema validation.

## 2. Core Components
### A. Non-AI Baseline (RegEx + Gazetteer)
- **Mechanism:** Searches for drug matches against a static dictionary (`["metformin", "amoxicillin", "lisinopri
- **Shortcomings:** Incapable of understanding negation or contextual boundaries (e.g., falsely extracts discont

### B. Gated LLM Pipeline (`gemini-1.5-flash`)
- **Semantic Extraction Layer:** System prompts the model to extract active medications with verbatim justificat
- **Rigid Output Schema:** Structured via Pydantic model (`MedicalSchema` containing `MedicationExtraction` obje
- **Deterministic Guardrail Layer:** Post-generation Python checker that verifies `verbatim_source_phrase` exist

## 3. Evaluation & Metrics (On Mock Consult Note)
- **Ground Truth:** `["Amoxicillin"]` (Metformin is discontinued; Diabetes belongs to Mother).
- **Performance Evaluation:**
  - **Baseline:** Precision: 0.50 | Recall: 1.00 | F1-Score: 0.67 (Fails due to False Positive extraction of neg
  - **AI System:** Precision: 1.00 | Recall: 1.00 | F1-Score: 1.00.
```

✓ Summary of Cell Execution Outputs

Below is the consolidated execution history and final verified outputs from all cells in the pipeline:

1. Library Installation & SDK Initialization (MsfcYq1iRy0r & 904060bb)

- **Status:** Successful
- **Stdout:** Client initialized successfully!

2. Clinical Note & Data Setup (NRqRkEl6SC1w & 7a7a24df)

- **Status:** Successful
- **Output Verification:**

✓ Variable 'mock_consult_note' is defined.
Content length: 314
Preview: \nCHIEF COMPLAINT: Patient presents with persistent...

3. Non-AI Baseline Extraction (GPvqLs9WSFuh)

- **Status:** Successful
- **Output:**

```
{  
  "Metformin": "500mg",  
  "Amoxicillin": "250mg"  
}
```

Observation: The baseline parser incorrectly extracted "Metformin" because it lacks structural context for negation checking.

4. Gated LLM Extraction Pipeline (x3ioB29LSJbG & 74e82926)

- **Model Used:** gemini-3.5-flash
- **Status:** Successful
- **Output:**

```
[  
  {  
    "drug_name": "Amoxicillin",  
    "dosage": "250mg TDS for 7 days",
```

```
"verbatim_source_phrase": "Amoxicillin 250mg TDS for 7 days"
}
```

Observation: The Gated LLM successfully ignored both discontinued medications (Metformin) and family medical history (Mother's Diabetes).

5. Quantitative Evaluation Metrics (9daecaa9 & 61ca62ed)

- Ground Truth: ["Amoxicillin"]
- Output Metrics:
 - Baseline System: {'Precision': 0.5, 'Recall': 1.0, 'F1-Score': 0.67}
 - Gated AI System: {'Precision': 1.0, 'Recall': 1.0, 'F1-Score': 1.0}

TT B I <> ↺ 🖼️ “ ≡ ≡ — ψ 😊 ☰ Close

Comparative Evaluation Metrics Summary			
Metric	Non-AI Baseline (Regex)	Gated AI System (Gemini 3.5 Flash)	
Precision	0.50	1.00	
Recall	1.00	1.00	
F1-Score	0.67	1.00	
Key Insights:			
Precision Improvement (+50%): The Non-AI Baseline falsely extracted Metformin (which was explicitly discontinued in the medical note). The Gated AI system correctly identified that Metformin should not be extracted, achieving a perfect precision score of 1.00.			
Recall Stability (1.00): Both systems successfully captured the active medication Amoxicillin.			
F1-Score Improvement (+33%): Due to eliminating false positives through semantic reasoning, the			

Comparative Evaluation Metrics Summary Metric Non-AI Baseline (Regex) Gated AI System (Gemini 3.5 Flash) Precision 0.50 1.00 Recall 1.00 1.00 F1-Score 0.67 1.00 Key Insights: Precision Improvement (+50%): The Non-AI Baseline falsely extracted Metformin (which was explicitly discontinued in the medical note). The Gated AI system correctly identified that Metformin should not be extracted, achieving a perfect precision score of 1.00. Recall Stability (1.00): Both systems successfully captured the active medication Amoxicillin. F1-Score Improvement (+33%): Due to eliminating false positives through semantic reasoning, the overall F1-score of the AI system reached a perfect 1.00.

overall F1-score of the AI system reached a perfect 1.00.